

HLD Case Study 7: Proximity Service (Yelp / Google Maps Places)

Key Focus Areas: Geospatial indexing algorithms, Geohash vs Google S2 vs Uber H3, Quadrees, and low-latency nearest-neighbor search.

1. Problem Statement & Functional Requirements

Design a global proximity service capable of answering spatial queries like *"Find the top-rated 20 restaurants within 5 km of my current GPS location"* in under 50 milliseconds.

Requirements:

1. Fast nearest-neighbor query by radius ($R \text{ km}$) and category.
2. Business owners can add or update location metadata and operating hours.
3. Scale to 500 million active users and 100 million places worldwide.
4. Support rapid geographic panning and zoom levels on mobile maps.

2. Geospatial Indexing: Why Standard Databases Fail

A standard SQL query attempting to find locations within a radius:

```
SELECT * FROM places
WHERE latitude BETWEEN 37.70 AND 37.80
      AND longitude BETWEEN -122.50 AND -122.40;
```

Even with a 2D composite index `(latitude, longitude)`, B-Trees can only effectively filter on **one dimension at a time**. The database scans millions of candidate rows along the latitude slice and filters longitude manually in memory, causing severe latency degradation.

3. Spatial Indexing Algorithms Compared

Algorithm	Representation	Neighbor Distance	Edge Artifacts	Industry Adoption
Geohash	Rectangular base32 grid strings	Unequal (orthogonal vs diagonal neighbors)	Boundary discontinuity issues	Redis GEO, DynamoDB, MongoDB
QuadTree	Hierarchical 4-way tree in memory	Variable	Memory fragmentation across node splits	In-memory spatial caches
Google S2	Hilbert curve on projected cube	Uniform Hilbert distance	Minimal distortion at poles	Google Maps, Foursquare
Uber H3	Hexagonal Hierarchical Grid	Identical (All 6 neighbors are strictly equidistant!)	Zero diagonal distortion	Uber, Lyft, DoorDash

```
flowchart LR
    Square["Square Grid (Geohash)"] --> Orth["Orthogonal neighbor = D"]
    Square --> Diag["Diagonal neighbor = sqrt(2)*D"]
    Hexagon["Hexagonal Grid (Uber H3)"] --> Identical["All 6 neighboring hexagons share identical distance"]
```

4. High-Level Architecture Diagram

```
flowchart TD
    Client["User Mobile App (GPS Lat, Lon)"] --> LB["Global Load Balancer"]
    LB --> GeoAPI["Proximity Query Service"]
    GeoAPI --> SpatialIndex["Distributed Spatial Index (Redis Cluster / ScyllaDB)"]
    SpatialIndex --> PlaceCache["Places Metadata Cache (Memcached/Redis)"]
    GeoAPI --> PlaceMetadataDB["Permanent Places DB (PostgreSQL / PostGIS)"]
```

5. The Geohash / H3 Query Walkthrough

```
sequenceDiagram
    autonumber
    actor User as User App
    participant Geo as Proximity Service
    participant Index as Redis Spatial Index
    participant Cache as Metadata Cache

    User->>Geo: search_nearby(lat=37.7749, lon=-122.4194, radius_km=2.0)
    Note over Geo: 1. Convert (lat, lon) to central H3 Hexagon / Geohash
    Note over Geo: 2. Calculate the 6 surrounding neighbor hex cells (Coverage ring)
    Geo->>Index: SMEMBERS [Center_Hex, Neighbor_1, ..., Neighbor_6]
    Index->>Geo: Returns list of 150 place_ids
    Geo->>Cache: MGET place_ids
    Cache->>Geo: Returns full place details (Ratings, photos)
    Note over Geo: 3. Filter by radius, sort by rating, paginate top 20
    Geo->>User: 200 OK (20 nearby venues in 18ms!)
```